

MES Maintenance Module

Database Schema & Data-Flow Reference

Toyota Boshoku Device India — Phase 2 EOL MES | PostgreSQL @ 192.168.10.210 / energydb

Scope: Every PostgreSQL table that participates in the Maintenance workflow — from the moment a line is flagged BREAKDOWN to CAPA closure. For every column we document: what the column physically stores, who writes it, and which UI / endpoint triggers the write.

Maintenance state machine (mes_breakdowns.state):

OPEN → RESOLVED → CLOSED

- **OPEN:** collector / API created the row; the line is currently down
- **RESOLVED:** ended_at stamped — line back up, awaiting slip closure
- **CLOSED:** Maintenance has submitted the closure form

Document layout: One section per table. Tables with JSONB payload columns get a dedicated sub-section documenting every nested key observed in production data.

Tables Covered

Table	Role
mes_breakdowns	Master breakdown record – one row per line stoppage. Holds OPEN/RESOLVED/CLOSED state, production-half slip, maintenance-half slip.
mes_breakdown_slip_config	Tunables: how long Production / Maintenance have to fill their half before a slip is auto-raised.
mes_breakdown_mail_levels	Escalation matrix: who gets a mail when a breakdown has been open for N minutes.
mes_breakdown_mail_log	Audit log: every escalation email actually sent (success or failure).
mes_capa_thresholds	Per-line / per-machine CAPA trigger limits (single-event & monthly cumulative).
mes_capa	CAPA filings – opened either auto from threshold breach or manually by Maintenance.
pm_check_master	Preventive-maintenance check definitions (daily / weekly / monthly checklists per machine type).
repair_work	Workshop repair work-orders – multi-step repairs tied back to a breakdown ticket.

1. mes_breakdowns

Primary table for the Maintenance Dashboard. The collector and the Breakdown ANDON both create rows here; the Breakdown Slip form fills **production_data** and **maintenance_data** JSONBs in two passes.

Writer: routers/breakdowns.py (open / resolve / production-fill / close). Read by: Maintenance dashboard, ANDON live tile, CAPA pending list, KPI router.

Column	Type	Stores
id	int (PK)	Auto-increment breakdown ID.
line_id	int (FK lines)	Which production line went down.
zone_id	int (FK zones)	Optional plant zone; copied from the line at row creation.
shift_name	varchar	Shift the breakdown started in (resolved from shift_configs via started_at).
serial_in_shift	int	1-based counter within (line_id, shift) — first breakdown of the shift is 1.
started_at	timestamp NOT NULL	Wall-clock when the line went BREAKDOWN. Defaults NOW().
ended_at	timestamp NULL	Wall-clock when the line returned to RUNNING. NULL while state='OPEN'.
state	varchar NOT NULL	OPEN / RESOLVED / CLOSED. Default OPEN.
reason	text	Free-text reason; usually 'BREAKDOWN' (auto) or whatever the operator typed in ANDON.
opened_by_user_id	int (FK users)	User who clicked 'Raise Breakdown' on ANDON; NULL if collector auto-opened it.
closed_by_user_id	int (FK users)	Maintenance user who submitted the closure form.
production_data	jsonb	Production half of the slip. See sub-table on the next page.
production_filled_at	timestamp	Set when Production submits their half via POST /api/breakdowns/{id}/production-fill.
production_filled_by_user_id	int (FK users)	Who filled the Production half.
maintenance_data	jsonb	Maintenance half of the slip. See sub-table on the next page.
maintenance_filled_at	timestamp	Set when Maintenance submits the closure form.
maintenance_filled_by_user_id	int (FK users)	Who filled the Maintenance half.
closure_data	jsonb	Legacy alias for maintenance_data. Old callers still POST 'closure_data'; the close endpoint copies it into maintenance_data and keeps a duplicate here for audit.
closed_at	timestamp	Set when the close endpoint transitions state to CLOSED.
created_at	timestamp	Row creation NOW(); same as started_at unless backfilled.
updated_at	timestamp	Bumped on every UPDATE.

1a. mes_breakdowns.production_data — JSONB keys

Filled by Production / line leader via the Breakdown Slip dialog (left half). Source: Maintenance dashboard → 'Fill Production Slip' button.

Key	Stores
date	Slip date (yyyy-mm-dd). Defaults to today.
shift	Shift label (A / B / C).
line	Line name as shown to operator (e.g. 'YNC L6').
zone	Plant zone the line sits in.
category	Tick-box category: Mechanical / Electrical / Quality / Other.
model_no	Model number being produced when breakdown happened.
machine_no	Sub-machine number (e.g. M3) — matches machines.machine_no.
machine_name	Human-readable sub-machine name (e.g. 'Lock Bar Insert').

Key	Stores
machine_operator_name	Operator running the machine at the time.
line_leader_name	Line leader on duty.
bd_received_time	Time Production formally received the breakdown notification.
problem_reported_by_production	Free-text: how Production described the symptom.

1b. mes_breakdowns.maintenance_data — JSONB keys

Filled by Maintenance via the closure form after the line is back up. Source: Maintenance dashboard → 'Close Breakdown' → closure dialog.

Key	Stores
actual_problem_observed	Free-text: what Maintenance actually found on inspection.
action_taken_on_problem	Free-text: corrective action performed.
spares_used	Free-text: list of spare parts consumed (part numbers / names).
bd_attended_by	Maintenance technician name who responded.
problem_related_to	Object {tool_room: bool, maintenance: bool} — which group owns the root cause.
received_by	Object {name, sign} — Maintenance shift in-charge sign-off.
prepared_by	Object {name, sign} — technician who prepared the slip.
quality_engineer	Object {name, sign} — QA sign-off (required for Quality-category breakdowns).
line_leader_operator	Object {name, sign} — Production countersignature confirming the line is back up.

1c. Auto-computed fields (never editable on slip)

The four header fields on the printed slip are stamped by the system — Production / Maintenance cannot edit them:

Slip field	Source
B/D Start Date + Start Time	started_at (collector or ANDON)
B/D OK Date + OK Time	ended_at (collector when line resumes RUNNING)
M/C Down Time (minutes)	Computed: (ended_at – started_at) / 60
Serial No. for the shift	serial_in_shift — incremented per (line_id, shift_name) at row creation

2. mes_breakdown_slip_config

Single-row global tunables for the slip workflow. Default scope is GLOBAL.

Writer: admin via Maintenance Settings UI. Read by: `routers/breakdowns.py` when evaluating whether to raise a 'slip overdue' flag and when computing escalation timeouts.

Column	Type	Stores
scope	text NOT NULL	Always 'GLOBAL' today; reserved for future per-line scoping.
production_fill_timeout_min	int NOT NULL	How many minutes Production has from breakdown start to fill their half before flagged overdue. Default 15.
maintenance_fill_timeout_min	int NOT NULL	How many minutes Maintenance has from RESOLVED to submit closure form. Default 60.
slip_raise_threshold_min	int NOT NULL	Minimum breakdown duration before a slip is even required. Quick blips below this don't generate paperwork. Default 10.
updated_at	timestampz NOT NULL	Last admin change.

3. mes_breakdown_mail_levels

Escalation matrix. One row per escalation tier (Level 1, Level 2, ...). The mailer scans this table every minute — when a breakdown crosses **delay_minutes** and no mail has been logged for that (breakdown, level) pair, it sends one.

Writer: `routers/breakdown_mail.py` (admin CRUD). Read by: the periodic mail worker.

Column	Type	Stores
id	int (PK)	Auto-increment.
level_no	int NOT NULL	1 = first escalation, 2 = second, etc.
label	varchar	Display name (e.g. 'Shift Engineer', 'Plant Head').
delay_minutes	int NOT NULL	How many minutes the breakdown must be open before this level fires.
to_addresses	text NOT NULL	Comma-separated TO recipients (e.g. 'eng1@x.com, eng2@x.com').
cc_addresses	text NOT NULL	Comma-separated CC recipients.
is_active	bool NOT NULL	Disable a level without deleting it.
created_at / updated_at	timestamp	Standard audit columns.

4. mes_breakdown_mail_log

Audit trail: one row per email actually attempted. Acts as the duplicate-suppression key — if a row exists for (breakdown_id, level_id), the mailer skips it.

Writer: the mail worker in `routers/breakdown_mail.py`. Read by: admin reports / audit UI.

Column	Type	Stores
id	int (PK)	Auto-increment.
breakdown_id	int NOT NULL	FK → <code>mes_breakdowns.id</code> .
level_id	int NOT NULL	FK → <code>mes_breakdown_mail_levels.id</code> .
sent_at	timestamp NOT NULL	When the send was attempted. Default NOW().
status	varchar NOT NULL	OK on success, FAIL on SMTP error. Default OK.
to_addresses	text	Actual recipient list at send time (snapshot of <code>levels.to_addresses</code>).
cc_addresses	text	Actual CC list at send time.
error	text	SMTP error string when status='FAIL'. NULL on success.

5. mes_capa_thresholds

Defines when a CAPA must be opened. Lookup is most-specific-wins:

Machine (line_id + machine_no) → Line (line_id) → Global

Writer: admin via Maintenance Settings / CAPA Thresholds page. Read by: routers/capa.py /pending endpoint and CAPA trigger detector.

Column	Type	Stores
id	int (PK)	Auto-increment.
scope	varchar NOT NULL	GLOBAL / LINE / MACHINE.
line_id	int	FK → lines.id. Required for LINE / MACHINE scope; NULL for GLOBAL.
machine_no	varchar	Required for MACHINE scope (e.g. 'M3'); NULL otherwise.
label	varchar	Optional admin-friendly nickname.
monthly_sum_minutes_limit	int NOT NULL	Trigger CAPA when month-to-date breakdown minutes for this scope cross this. Default 120.
single_breakdown_minutes_limit	int NOT NULL	Trigger CAPA when a single breakdown exceeds this. Default 60.
pareto_pct	int NOT NULL	Pareto cutoff (% of total downtime) used to highlight top-offender machines on the CAPA dashboard. Default 80.
is_active	bool NOT NULL	Disable without deleting.
created_at / updated_at	timestamp	Standard audit columns.

6. mes_capa

CAPA filings. Either auto-detected by the threshold engine (**trigger_kind** SINGLE_LIMIT / MONTHLY_LIMIT) or opened manually by Maintenance from a breakdown.

Writer: routers/capa.py (POST /api/capa create, PUT update, POST /{id}/close). Read by: CAPA dashboard, Maintenance KPI router.

Column	Type	Stores
id	int (PK)	Auto-increment.
breakdown_id	int	FK → mes_breakdowns.id. NULL for purely cumulative-monthly CAPAs.
trigger_kind	varchar NOT NULL	SINGLE_LIMIT (one big breakdown) or MONTHLY_LIMIT (cumulative).
trigger_value_minutes	int	The actual measured value at the moment the CAPA was raised (e.g. 145 minutes for a 145-min breakdown).
threshold_minutes	int	The applicable limit at the time (snapshot of the threshold row).
line_id, line_name	int / varchar	Line snapshot — copied at open time so a renamed line doesn't break the CAPA report.
zone_id, zone_name	int / varchar	Plant zone snapshot.
machine_no, machine_name	varchar	Sub-machine snapshot.
month_year	char(7)	'YYYY-MM' bucket for monthly-trigger CAPAs (so the same machine can have one per month).
status	varchar NOT NULL	OPEN / IN_PROGRESS / CLOSED.
capa_data	jsonb	Full 8D-style CAPA payload. See sub-table below.
opened_by_user_id / opened_at	int / timestamp	Who opened the CAPA and when.
closed_by_user_id / closed_at	int / timestamp	Closer + timestamp.
updated_at	timestamp	Bumped on every update.

6a. mes_capa.capa_data — JSONB keys

8D-style CAPA payload. Sensible defaults are shipped by the API at creation; Maintenance refines them in the CAPA dialog.

Key	Stores
problem_description	What went wrong (1-line summary used on dashboards).
root_cause	Root cause analysis (5-Why output).
containment	Containment / immediate action taken.
corrective_action	Permanent corrective action.
preventive_action	Preventive action to stop recurrence.
verification	How effectiveness will be verified (audit plan).
responsible	Person / team accountable for closure.
target_date	Committed closure date (yyyy-mm-dd).
sign_offs	Object — counter-signature blocks (e.g. {maintenance_head, qa_head, plant_head}).

7. pm_check_master

Master list of preventive-maintenance check items. Each row = one checklist line that has to be ticked at the given frequency.

Writer: admin via PM Master UI. Read by: PM execution screen (renders the checklist for the day).

Column	Type	Stores
id	int (PK)	Auto-increment.
check_type	varchar NOT NULL	Category bucket — DAILY / WEEKLY / MONTHLY / QUARTERLY etc.
check_name	varchar NOT NULL	Short checklist line ('Inspect drive belt tension').
description	text	Detailed how-to (acceptance criteria, tool required, photo reference).
frequency_count	int NOT NULL	How many times in the check_type window. e.g. DAILY + frequency_count=2 → twice a day.
is_active	bool	Disable a check without deleting it.
created_at / updated_at	timestamp	Standard audit columns.

8. repair_work

Detailed multi-step repair log tied back to a breakdown ticket. One breakdown can have many repair_work rows when the fix took several attempts or several parts swaps.

Writer: Maintenance workshop UI / repair-log endpoint. Read by: Repair history view, MTTR drill-down on KPI dashboard.

Column	Type	Stores
id	int (PK)	Auto-increment.
ticket_id	int	FK → mes_breakdowns.id (the parent breakdown).
work_sequence	int NOT NULL	1-based step number within the same ticket. Step 2 = second attempt / second part swap.
root_cause	text	Step-level diagnosis (may differ from CAPA-level root cause).
action_taken	text	What the technician did in this step.
spare_parts_used	text	Free-text or CSV of part numbers consumed in this step.
downtime_minutes	int	Minutes attributable to this step (sum across steps ~= total breakdown duration).
before_repair_image	varchar(path)	Path / URL of 'before' photo (uploaded via multipart).
after_repair_image	varchar(path)	Path / URL of 'after' photo.
performed_by	varchar	Technician name / username.
performed_at	timestamp	When this step happened. Default NOW().
work_status	varchar	COMPLETED (default) / IN_PROGRESS / FAILED.
created_at	timestamp	Row creation.

Data Flow Cheat-Sheet

End-to-end path of a single breakdown event through every maintenance table:

#	Event	Table.column written
1	Collector detects line went IDLE→BREAKDOWN, OR operator clicks ANDON 'Raise Breakdown'	mes_breakdowns: id, line_id, zone_id, shift_name, serial_in_shift, started_at, state='OPEN', reason, opened_by_user_id
2	Slip-config timer expires (15 min default) without Production filling slip	mes_breakdown_mail_log: row inserted at every mail level whose delay_minutes ≤ elapsed
3	Production submits their half from the slip dialog	mes_breakdowns: production_data, production_filled_at, production_filled_by_user_id
4	Collector detects line back to RUNNING (or Maintenance clicks 'Resolve' on dashboard)	mes_breakdowns: ended_at, state='RESOLVED'
5	Maintenance submits closure form	mes_breakdowns: maintenance_data, closure_data (legacy duplicate), maintenance_filled_at, maintenance_filled_by_user_id, closed_at, closed_by_user_id, state='CLOSED'
6	(If multi-step repair) workshop logs each attempt	repair_work: one row per attempt, all sharing ticket_id = breakdown_id
7	CAPA detector finds breakdown's downtime crossed single_breakdown_minutes_limit, OR month-to-date sum crossed monthly_sum_minutes_limit	mes_capa: new row with trigger_kind, trigger_value_minutes, threshold_minutes, status='OPEN', capa_data with defaults
8	Maintenance refines CAPA in the 8D dialog	mes_capa: capa_data updated, status='IN_PROGRESS' or 'CLOSED'

KPI derivations (read-only)

routers/maintenance_kpi.py computes these from the tables above — no separate KPI table:

KPI	Formula
MTBF	$(\text{uptime_minutes_in_window}) / (\text{count of breakdowns closed in window})$. uptime = window_minutes – SUM(downtime). Zone- and line-level.
MTTR	AVG(ended_at – started_at) over CLOSED breakdowns in window.
LTTR	MAX(ended_at – started_at) — longest time to repair.
LTBF	MAX gap between consecutive breakdowns (longest time between failures).
Slip overdue count	Count of OPEN/RESOLVED rows where (NOW – started_at) > production_fill_timeout_min and production_data IS NULL.
CAPA aging	Count of mes_capa rows where status ≠ 'CLOSED' grouped by (NOW – opened_at) bucket.